Sprint 4 – Technical Report

Team 8's Submission for Sprint #4

Team Name: Wolf of The 7th Street

Pipeline Contract and Stage Map

Pipeline Contract

1. Project question addressed by this pipeline

Our final project question is whether historical stock-market information can be used to identify patterns and predict whether a stock price will increase on the next trading day.

The Sprint 4 PySpark pipeline prepares minute-level stock data into daily stock data. It also reproduces the monthly stock-price increase analysis from Sprint 3, performs a second monthly analysis using average daily percentage change, and prepares the data for a machine-learning model in Sprint 5.

2. How the result contributes to the final project

The results contribute to the final project in several ways:

- Minute-level observations are converted into daily stock records.
- Daily stock features are created for machine-learning use.
- The PA3 monthly increase-ratio analysis is reproduced and verified.
- A second analysis investigates whether stock behavior changes by calendar month.
- Training and testing datasets are prepared for the Sprint 5 prediction model.
- PySpark and DuckDB implementations are compared for correctness and speed.

3. Raw input file, delimiter, header status, and fields used

Raw input: APOLLOHOSP minute-level stock data from the PA3 sample.

Delimiter: Tab-separated values.

Input size: 1,033,587 rows.

Fields used:

- name
- date
- open
- high
- low
- close
- volume
- price_change
- volume_change

The sample data was used to prepare and test the PySpark pipeline before applying the same process to larger data.

4. Expected schema of the daily analysis-ready table

One row represents one stock on one trading day.

The daily table uses (name, date) as its natural key.

Column	Data Type	Description
name	String	Stock ticker symbol
date	Date	Trading date
open	Double	Daily opening price
high	Double	Highest price during the day
low	Double	Lowest price during the day
close	Double	Daily closing price
price_change	String	Whether the stock increased compared with the previous day
change_pc	Double	Daily percentage range calculated as (high - low) / open

5. Derived variables

The pipeline creates the following derived variables:

change_pc:

```
None
(high - low) / open
```

price_change:

Today’s date is compared to yesterday’s date in order to see if the price has gone up or down.

Since there was no previous date for the first date, its price change value is assigned a zero.

6. Rules

Filter:
Only rows with increased/decreased prices are selected for Analysis 1. Rows with price_change = None are ignored while computing comparable rows.

Missing values:
Rows with missing values in the price_change field are kept while preparing the data but not used in the increase ratio calculation.

Grouping:
The daily stock record is grouped by month of the calendar year for both analyses.

Ordering:
The results of the analysis are ordered by month from 1 to 12.

Join:
Analysis 2 makes use of a month look up table to append the month name and quarter. The look up keys are unique. There are no unmatched daily rows and the total row count stayed at 2,769 before and after the join.

Window:
Previous day comparison is used to calculate price_change field. The window is ordered by trading date. The first row does not have any previous row and hence its value is set as 0.

Stage Map

```
None
Minute-level input data
        ↓
Read and inspect schema
        ↓
Cast fields and validate values
        ↓
Aggregate minute records into daily records
        ↓
Create change_pc and price_change
        ↓
Save prepared data as Parquet
        ↓
Read Parquet back and verify schema/count
        ↓
Analysis 1: Monthly increase ratio
        ↓
Analysis 2: Monthly average change_pc
        ↓
Create training, testing, and development datasets
        ↓
Compare PySpark and DuckDB benchmark results
        ↓
Sprint 5 machine-learning handoff
```

Data Preparation and Data Quality Assessment

In the PySpark notebook, the data was loaded with minute granularity, checked for schema, selected relevant columns, cast to the correct data type, and aggregated by trade date.

The resulting daily table had 2,769 rows.

Following are the quality assessments performed:

- Minute rows read: 1,033,587
- Minute rows rejected: 0
- Valid minute rows: 1,033,587
- Rows with null model values: 0

- Invalid negative values: 0
- Daily rows before domain filtering: 2,769
- Daily rows after domain filtering: 2,769
- Date key uniqueness: True
- (name, date) uniqueness: True
- Duplicate month lookup keys: 0
- Unmatched daily rows after the month join: 0
- Parquet read-back row count: 2,769
- Parquet schema verification: True
- low <= high: True
- low <= open <= high: True
- low <= close <= high: True

PA3 Validation rules did not accept rows with zero minutes. In total, there were 73,697 rows with price_change = None, and they were not included in the first analysis. There were 959,890 rows that had increased/decreased labels.

Numbers of issues were counted separately since some data-quality criteria may have overlapped, hence, numbers cannot be summed up as though those are totally separate rows.

Parquet Creation Process

The prepared daily dataset was saved as Parquet under:

```
None
pa4/data/prepared/daily_full.parquet
```

The training dataset was saved under:

```
None
pa4/data/prepared/train.parquet
```

The testing dataset was saved under:

```
None
pa4/data/prepared/test.parquet
```

The development sample was saved under:

```
None
pa4/data/prepared/dev_sample_train.parquet
```

The Parquet files were read back into PySpark. The row counts and schemas were verified after reading.

The following read-back checks passed:

- Daily Parquet row count: 2,769
- Training Parquet row count: 2,129
- Testing Parquet row count: 533
- Development sample row count: 397
- Audit table row count: 2,663
- All read-back schemas matched the original schemas
- Training up-day total before and after read-back: 1,075 / 1,075

Task 2: Analytical Questions

Analysis 1: Month-on-Month Growth Rate

The first analysis was a repeat of the Sprint 3 month-on-month growth rate analysis, using the identical inputs, filters, groupings, and missing value handling approach.

The analysis involved 1,033,587 minute-level rows.

The outputs were:

- Rejected rows: 0
- Valid rows: 1,033,587
- Rows with price_change = None: 73,697
- Comparable rows: 959,890
- Monthly results matched: 12 out of 12
- Ratio range: 0.4950 to 0.5023

Month	Increase Count	Comparable Rows	Increase Ratio
1	40,634	81,372	0.4994

2	41,969	84,465	0.4969
3	42,240	84,758	0.4984
4	37,039	73,972	0.5007
5	39,971	80,460	0.4968
6	40,141	80,905	0.4961
7	41,138	81,898	0.5023
8	39,778	80,091	0.4967
9	39,154	78,564	0.4984
10	37,761	75,986	0.4969
11	37,787	75,913	0.4978
12	40,344	81,506	0.4950

Analysis 1 Finding

All monthly increase ratios were near 0.50. It means that the monthly increase outcome was nearly a coin toss.

The study did not present compelling data on a particular month which generates more price increases. Nonetheless, the task was accomplished by confirming that the PySpark solution accurately mimicked the PA3 solution.

Analysis 2: Average Daily Change by Calendar Month

Analysis 2 calculated the average daily change_pc for each calendar month using 2,769 daily observations.Analysis 2 Finding

Month	Month Name	Quarter	Days	Average Change	Standard Deviation	Up-Day Percentage	Rank
1	January	Q1	234	0.0724	1.7342	0.5171	7

2	February	Q1	240	0.1092	2.3017	0.4896	6
3	March	Q1	242	-0.2620	2.3688	0.4504	12
4	April	Q2	212	0.1125	2.1067	0.4906	5
5	May	Q2	233	-0.0729	1.8117	0.4592	11
6	June	Q2	232	0.2568	1.8389	0.5259	1
7	July	Q3	240	0.2308	1.5381	0.5542	3
8	August	Q3	229	0.1717	2.3058	0.4629	4
9	September	Q3	226	0.0406	1.8511	0.4779	9
10	October	Q4	223	0.0229	1.5632	0.5336	10
11	November	Q4	222	0.2568	2.2363	0.4775	1
12	December	Q4	235	0.0439	1.6050	0.5106	8

June and November had the highest average daily change at approximately +0.26%.

The lowest average change per day was recorded in March, amounting to -0.26%.

The standard deviations varied from 1.5% to 2.4% and were much higher than the differences between the monthly averages.

Hence, there was no clear evidence that the seasonal behavior was stable.

The diagram for this test is stored as:

```
None
pa4/docs/analysis2_avg_change_by_month.png
```

Task 3: Sampling and Final-Sprint Handoff

The group chose Path B, prediction preparation.

The target variable is target_up, showing if the stock price has risen in the next trading day.

The prediction is made at the end of the current trading day. Fields disclosing outcomes are not included in the feature list.

Features List

The feature list includes:

```
None
ret_1d
ret_lag1
ret_lag2
ret_mean_90
ret_std_90
range_pct
vol_ratio_20
month
dow
```

Train/Test Split

The group generated 2,663 rows with labels.

These rows were removed:

- 1 row for the final day because there was no next day
- 16 rows with a flat next closing price
- 89 warm-up rows missing enough history for the 90-day and 20-day features

The time split produced:

- Training rows: 2,129
- Purged boundary row: 1
- Testing rows: 533

The test dates started with February 7, 2024. The condition to check that the latest training date is earlier than the earliest testing date was successful.

Development Sample

The development sample was derived from the training data alone.

Sampling method:

```
None
sample(withReplacement=False, fraction=0.2, seed=131)
```

The training population contained 2,129 rows. The development sample contained 397 rows, representing 18.6% of the training data.

Target Group	Source Count	Source Percentage	Sample Count	Sample Percentage
Down / 0	1,054	49.51%	193	48.61%
Up / 1	1,075	50.49%	204	51.39%

The source and sample target proportions were similar. Year proportions changed by no more than 1.80 percentage points.

Handoff Outputs

The prepared outputs were saved in persistent team storage:

```
None
pa4/data/prepared/daily_full.parquet
pa4/data/prepared/train.parquet
pa4/data/prepared/test.parquet
pa4/data/prepared/dev_sample_train.parquet
pa4/out/analysis_1.csv
pa4/out/analysis_2.csv
pa4/out/quality_checks.csv
pa4/out/sample_comparison.csv
pa4/out/benchmark.csv
```

The daily Parquet file contains 2,769 rows.

The training Parquet file contains 2,129 rows.

The testing Parquet file contains 533 rows.

The development sample contains 397 rows.

Both analysis files contain 12 monthly result rows.

Task 4: PySpark and DuckDB Benchmark

Task 4 compared the PA3 monthly analysis using PySpark and DuckDB.

Both tools used:

- The same input records
- The same TSV input format
- The same filtering rules
- The same grouping rules
- The same ordering
- The same numeric verification rules

The benchmark input contained 1,033,587 rows and was 87,079,423 bytes.

Benchmark ResultsBenchmark Summary

Implementation	Environment	Input Rows	Input Bytes	Format	Run	Time	Output Rows	Verified
PySpark	Colab	1,033,587	87,079,423	TSV	1	3.6132 sec	12	True
PySpark	Colab	1,033,587	87,079,423	TSV	2	2.8331 sec	12	True
PySpark	Colab	1,033,587	87,079,423	TSV	3	2.3469 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	1	0.6216 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	2	0.5415 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	3	0.5593 sec	12	True

DuckDB results:

- Mean: 0.574133 seconds

- Median: 0.5593 seconds
- Standard deviation: 0.042060 seconds
- Minimum: 0.5415 seconds
- Maximum: 0.6216 seconds

PySpark results:

- Mean: 2.931067 seconds
- Median: 2.8331 seconds
- Standard deviation: 0.638809 seconds
- Minimum: 2.3469 seconds
- Maximum: 3.6132 seconds

Benchmark Interpretation

DuckDB was faster than PySpark for this particular Colab environment and time period.

The median of DuckDB was 0.5593 seconds, whereas the median of PySpark was 2.8331 seconds. Thus, PySpark was about 5.1 times slower than DuckDB in median execution time.

DuckDB also demonstrated less variability in execution.

In all six executions, 12 rows were produced and all of them were successful.

The tests reflect just one machine, one environment, and one workload. They are not sufficient to demonstrate that DuckDB always executes faster than Spark.

Failures Experienced and Diagnosis

There was an instance where one of the team members tried to push Task 4 from Google Colab. There was an authentication failure because the necessary username, email, and access token were not set.

It is advisable to download the entire repository on your computer and then push from an IDE or from the command line. It is also possible to set the GitHub access token and username in Google Colab.

There was also the question of whether to use the full data in Sprint 4 or just a sample. The team explained that Sprint 4 uses a stock sample to ensure the entire process is fine-tuned for Sprint 5. The same process can also be done on the full dataset using stock name as the filter.

Limitations

The analyses have been done with only one stock APOLLOHOSP. Thus, the conclusions drawn from here cannot automatically apply to all the stocks listed under the Nifty100 stocks.

The values of increase ratio per month were around 50%, and the difference between the average monthly change was small compared to the standard deviation of the monthly changes. Thus, there is no strong evidence of seasonality here.

The benchmarking is dependent on the hardware configuration, Colab environment, caching, version of software used, input format, and the timings defined.

Team Contributions and AI-Use Disclosure

Team Members

- Aydan Kaneshiro – Data Engineer
- Devarsh Shroff – Data Engineer
- Rishil Nilesh Patel – Data Storyteller
- Vakeesan Moorthy – Data Engineer
- Jeremy Lim – Project Manager

Team Contributions

- Vakeesan Moorthy: Completed Task 1 daily data preparation and derived variables.
- Devarsh Shroff: Completed Task 2 analytical questions and Task 3 sampling and final-sprint handoff.
- Aydan Kaneshiro: Completed Task 4 PySpark and DuckDB benchmark.
- Rishil Nilesh Patel: Prepared the Sprint 4 technical report and documentation.
- Jeremy Lim: Managed project coordination and sprint requirements.

AI-Usage Disclosure

AI was employed to develop potential features for the stock model, verify the syntax of PySpark, debug code and GitHub commands, and compile the report.

The team considered the suggestions, and was accountable for the submitted code, findings, and explanations, and the capacity to explain the project.